

Week 4: Authentication, Events & Messaging

Authentication, Event-Driven Systems & Messaging Internship Guide

Week 4: Authentication, Events & Messaging

The Complete Theoretical Intern Training & Implementation Guide

Welcome to the Week 4 Authentication, Events & Messaging Intern Guide. This week moves your cloud engineering skills into advanced enterprise design patterns: securing REST APIs, decoupled queuing services, fan-out event routing, and background compute processing on Amazon Web Services (AWS).

This guide serves as a self-contained manual. It contains no code snippets (except for step-by-step shell execution commands, file trees, and architecture diagrams), focusing instead on explaining every security token, message queue, pub/sub topic, state machine, and service integration from first principles. By reading this guide, you will understand exactly what was built during Week 4, how the components interact, and how to deploy and test the architectures step-by-step.

Table of Contents

1. Module 1: Authentication & Authorization (Cognito & JWT)
2. Module 2: Decoupled & Event-Driven Services (SQS, SNS & EventBridge)
3. Module 3: Asynchronous Compute Architectures
4. Module 4: Resiliency, Retries, and Dead-Letter Queues (DLQ)
5. Module 5: Workflow Orchestration & AWS Step Functions
6. Module 6: API Gateway Cognito Authorizers
7. Module 7: System Integrations: SQS Lambda Triggers
8. Module 8: The Decoupled Messaging & Image Processing Pipelines
9. Module 9: Terraform Resource Definitions for Messaging & Auth
10. Module 10: Hands-On Lab & Project Setup Guide
11. Module 11: Week 4 Learning Conclusion

Module 1: Authentication & Authorization (Cognito & JWT)

Securing microservices requires separating authorization from compute execution. Passing usernames and passwords in every request is insecure and creates bottlenecks.

1.1 JSON Web Tokens (JWT) Architecture

AWS Cognito and other identity providers represent user identity using cryptographically signed JSON Web Tokens (JWTs). A JWT is a base64-encoded string split into three segments separated by periods:

- Header: Declares the cryptographic algorithm used to sign the token (e.g., RS256) and the token type.
- Payload (Claims): Contains the actual user metadata assertions (claims) such as `email`, `sub` (unique user UUID), `exp` (expiration timestamp), and custom attributes like `custom:role`.
- Signature: Created by combining the base64-encoded Header and Payload with a private key. The signature guarantees that the contents of the header or payload have not been altered in transit.

1.2 Signature Verification Mechanics

When an API Gateway receives a request containing a JWT token, it validates the signature before routing it to the backend:

- The Gateway downloads the public JSON Web Key Set (JWKS) from Cognito.
- It decrypts the token's signature using the public key and checks that the token is not expired (`exp` claim is in the future).
- If the validation passes, the request is authorized. This allows the backend to trust the payload claims without running additional database queries.

Module 2: Decoupled & Event-Driven Services (SQS, SNS & EventBridge)

Decoupling is the practice of separating application modules so they can run and scale independently. This is achieved using three AWS messaging services:

2.1 Amazon SQS (Simple Queue Service)

SQS is a fully managed message queuing service. It uses a pull-based model where producers write messages to a queue, and consumers poll the queue to retrieve and process them.

- SQS acts as a buffer. If your backend is slow or goes offline, messages accumulate in the queue rather than failing. Once the backend recovers, it processes the backlog.
- Standard queues offer unlimited throughput and at-least-once message delivery.

2.2 Amazon SNS (Simple Notification Service)

SNS is a managed push-based pub/sub messaging service. A producer publishes a message to an SNS Topic, and SNS automatically distributes (fans out) that message to all active subscribers (e.g., SQS queues, Lambda functions, HTTP endpoints, or email addresses).

- Subscribers only receive messages published after their subscription is active.
- SNS is ideal for real-time notifications or routing the same event to multiple backend services simultaneously.

2.3 Amazon EventBridge

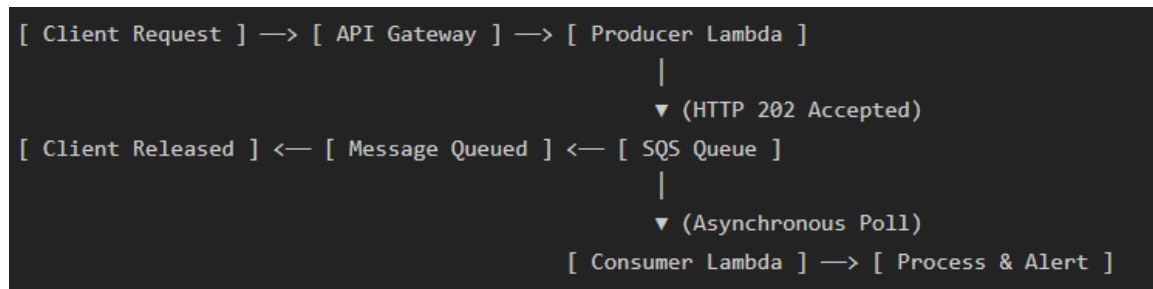
EventBridge is a serverless event bus. It enables you to route system events (such as S3 object changes or EC2 state changes) or custom application events to target services using rules.

- It supports cron-like schedule rules (via Amazon EventBridge Scheduler) to trigger Lambda functions or batch jobs at specific times.

Module 3: Asynchronous Compute Architectures

In traditional synchronous workflows, a client requests an action and blocks (waits) until the server completes the processing and returns a response. If the processing takes 10 seconds, the browser connection stays open.

In asynchronous compute architectures, the processing is split into a multi-step pipeline:



- Submission: The client sends a request. The API Gateway forwards it to a Producer Lambda function.
- Acceptance: The Producer Lambda validates the input, writes the request payload to an SQS queue, and instantly returns a `202 Accepted` status code. The client connection is released.
- Decoupled Processing: The Consumer Lambda function is triggered by the queue in the background. It reads the message, processes the data, and sends a notification when finished.

Module 4: Resiliency, Retries, and Dead-Letter Queues (DLQ)

Asynchronous workflows must be designed to handle failures gracefully.

4.1 Retry Policies & Poison Pill Payloads

- Retries: If a Consumer Lambda function crashes while processing a message, SQS keeps the message in the queue. After the Visibility Timeout expires, the message becomes visible again, and the Lambda retries the execution.
- Poison Pill: A message that contains bad data or formatting that causes the Lambda to crash every time it tries to process it. Without safety controls, this message would loop infinitely, blocking the queue and consuming execution costs.

4.2 Dead-Letter Queues (DLQ)

A Dead-Letter Queue (DLQ) is a secondary SQS queue configured to capture failed messages.

- You configure a Redrive Policy on the source queue specifying a `maxReceiveCount` (e.g., 3).
- If a message fails execution 3 times, SQS moves it to the DLQ. This keeps the main queue clear and allows developers to inspect the failed messages in the DLQ for debugging.

Module 5: Workflow Orchestration & AWS Step Functions

When an asynchronous workflow requires multiple sequential or parallel steps, managing retry logic and states within Lambda code becomes complex. AWS Step Functions solves this by letting you define workflows as State Machines.

- Workflows are defined using JSON-based Amazon States Language (ASL).
- It orchestrates multiple services (Lambda, SQS, DynamoDB) using states:
- Task State: Runs a compute task (like invoking a Lambda).
- Choice State: Evaluates input variables to branch the workflow path.
- Wait State: Pauses execution for a specified duration.
- Parallel State: Executes multiple branches simultaneously.

Module 6: API Gateway Cognito Authorizers

To secure your REST endpoints, you integrate API Gateway with Cognito using a Cognito Authorizer:

- When a client signs in via Cognito, it receives an ID Token (validating user identity) and an Access Token (authorizing specific operations).
- The client includes this token in the `Authorization` header of its API requests.

- API Gateway automatically validates the token signature against Cognito's public keys. If valid, the Gateway extracts the user claims (like their email) and passes them to the backend Lambda in the `requestContext.authorizer.claims` payload. If invalid, the Gateway blocks the request, returning a `401 Unauthorized` response.

Module 7: System Integrations: SQS Lambda Triggers

To process SQS queues using Lambda, AWS uses Event Source Mapping (ESM):

- AWS runs a poller service in the background that continuously queries the SQS queue for messages.
- When messages are found, the poller invokes the Lambda function, passing the messages in the `Records` list.
- Batch Size: The maximum number of messages sent to the Lambda in a single invocation (up to 10,000 for FIFO queues).
- Batch Window: The maximum time (in seconds) the poller waits to accumulate messages before invoking the Lambda.
- Visibility Timeout: The duration that SQS hides a message from other pollers while it is being processed. It must be set to at least 6 times the timeout of the consumer Lambda function to prevent double-processing.

Module 8: Decoupled Messaging & Image Processing Pipelines

This week's assignments feature two distinct decoupling architectures:

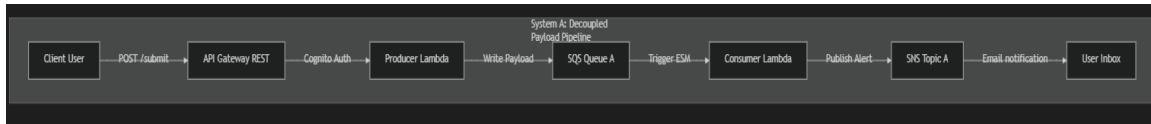
System A: Decoupled JSON Messaging Pipeline (Week 4 Root)

- Goal: Decouple general client JSON requests from background processing and notifications.
- Path:
 1. The client sends a request containing a JSON body to a protected API Gateway endpoint.

2. The Producer Lambda writes the JSON payload and user email to SQS, returning a `202 Accepted` response.

3. The SQS queue triggers the Consumer Lambda in the background.

4. The Consumer processes the payload and publishes a completion message to an SNS Topic, which emails the user.



System B: Decoupled Image Upload Pipeline ('Week 4/Image Uploads')

- Goal: Decouple S3 metadata extraction from heavy image processing.
- Path:

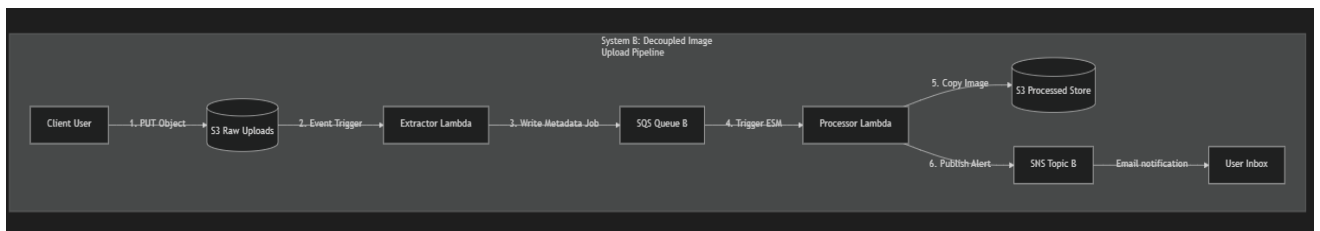
1. The client uploads an image directly to S3 using a pre-signed URL.

2. S3 triggers the Extractor Lambda in response to the upload event.

3. The Extractor reads the file metadata and sends a job message containing the S3 bucket name and key to an SQS Queue.

4. The SQS queue triggers the Processor Lambda.

5. The Processor copies the file to a processed bucket and publishes a status alert to an SNS Topic, emailing the user.



Module 9: Terraform Resource Definitions for Messaging & Auth

We configure the infrastructure using Terraform resources:

- `aws_cognito_user_pool`: Provisions the user pool directory.
- `aws_api_gateway_authorizer`: Links API Gateway to Cognito, specifying the target user pool ARNs.
- `aws_sqs_queue`: Creates the primary SQS queues.
- `aws_sqs_queue` with `redrive` policy: Configures Dead-Letter Queues (DLQs).
- `aws_sns_topic`: Creates the SNS notification topics.
- `aws_sns_topic_subscription`: Configures the email subscription list.
- `aws_lambda_event_source_mapping`: Connects SQS queues to Lambda triggers, enabling the background event pollers.

Module 10: Hands-On Lab & Project Setup Guide

This module details the directories, execution files, and configurations required to deploy and run the Week 4 serverless auth and messaging pipelines.

10.1 Project File Directory Structure

Below is the directory tree containing the code resources for this week's assignments:

```

c:\Users\Vaibhav.S.P\Downloads\TringApps Intern
└─ Week 4/
    │ main.tf                # System A: Auth, SQS & API Gateway configuration
    │ variables.tf           # System A: Variables config
    │ outputs.tf             # System A: Outputs declaration
    │ lambda/
    │ │ producer.py         # Producer function queuing payloads
    │ │ index.py            # Consumer function processing SQS to SNS
    └─ Image Uploads/
        │ main.tf           # System B: Full S3 OAC CF, SQS & SNS configurations
        │ variables.tf       # System B: Variables config
        │ outputs.tf         # System B: Outputs declaration
        │ index.html.tftpl   # Website dashboard code
        └─ lambda/
            │ presigned.py   # Pre-signed S3 url generator handler
            │ extractor.py   # S3 trigger metadata extractor handler
            └─ processor.py  # SQS trigger image copy & SNS handler

```

10.2 Step-by-Step Implementation Runbook

Follow these steps to deploy and test the components of Week 4:

Task A: Basic Tools & Credentials Setup

Verify your terminal is authenticated with your AWS account:

- Open your command terminal and navigate to the project directory:

```
cd "Week 4"
```

- Verify you have active AWS CLI credentials configured:

```
aws sts get-caller-identity
```

Task B: Simple Payload Messaging Pipeline (System A)

- Initialize the directory:

```
terraform init
```

- Validate the configuration files:

```
terraform validate
```

- Generate the plan:

terraform plan -out=tfplan

- Deploy the infrastructure to AWS:

terraform apply tfplan

- Verification: Capture the Cognito Pool ID, Client ID, and API Stage URL from the output variables.

Task C: Testing System A Decoupled Processing

- Log into your AWS Console and search for Cognito. Select your user pool.
- Manually register a user account, and confirm it.
- Use a tool (such as Postman or curl) to authenticate against Cognito, obtaining an ID Token.
- Send an HTTP POST request to your API Endpoint URL using the authorization token:

```
curl -X POST -H "Authorization: Bearer <ID_TOKEN>" -H "Content-Type: application/json" -d '{"task_name\': \'Decoupled Task\', \'priority\': \'high\'}' https://xxxx.execute-api.us-east-1.amazonaws.com/dev/submit
```

Expected Output: Returns immediately with status code `202 Accepted` and a unique SQS Message ID.

- Check your email inbox. Within 60 seconds, you should receive a notification containing the JSON payload details.
- Destroy the System A resources:

terraform destroy -auto-approve

Task D: Advanced S3 Image Upload Pipeline (System B)

To deploy the System B S3 image uploads pipeline:

- Navigate to the S3 Image Uploads folder:

```
cd "Image Uploads"
```

- Initialize and deploy:

```
terraform init
```

```
terraform plan -out=tfplan
```

```
terraform apply tfplan
```

- Verification: Take note of the output variables:
 1. ``api_url`` (The pre-signed generator API).
 2. ``cloudfront_url`` (The static website CDN URL).
 3. ``upload_bucket_name`` (The raw storage bucket).

Task E: Running the Frontend & Uploading Images

- Open the CloudFront URL in your browser.
- Sign up and log in using the web form.
- Select an image file (PNG or JPEG) and click Upload.
- The frontend will request a pre-signed URL, upload the file directly to the raw S3 bucket, and then wait.
- In the background:
 1. S3 triggers the Extractor Lambda.
 2. The Extractor writes a metadata job to the SQS Queue.
 3. The SQS Queue triggers the Processor Lambda.

4. The Processor Lambda copies the image to the processed S3 bucket and sends an email notification.
- Verification: Verify that the website dashboard status transitions to complete, and check your email inbox for the image metadata labels.

Task F: Teardown

When you are finished testing, destroy the resources to avoid ongoing AWS charges.

- Run the destroy command:

terraform destroy -auto-approve

Module 11: Week 4 Learning Conclusion

Throughout this authentication, event routing, and decoupled architectures week, we have progressed from basic serverless APIs to resilient event-driven architectures. Here is a summary of the core competencies and learning outcomes achieved:

- Token-Based Security: Mastered JWT token layouts (Header, Payload, Signature) and integrated Cognito Authorizers with API Gateway to secure endpoints.
- Asynchronous decoupling: Decoupled compute steps using SQS message queues to buffer high-traffic request bursts.
- Fan-Out Event Routing: Configured Amazon SNS to distribute event alerts to email subscribers.
- Background Compute ESM: Set up SQS Event Source Mappings (ESM) to trigger consumer Lambdas automatically in response to queue backlogs.
- System Resiliency: Configured Redrive Policies and Dead-Letter Queues (DLQ) to capture failed poison pill payloads.
- State Orchestration: Learned the fundamentals of AWS Step Functions to coordinate multi-step workflows.

By combining these decoupling patterns, you have built a secure, production-grade asynchronous pipeline, establishing a strong foundation for building scalable microservices!

Appendix A: Glossary of Week 4 Terminology

Acronym / Term	Full Name	Definition / Cloud Role
SQS	Simple Queue Service	A managed, pull-based message queuing service used to buffer and decouple system operations.
SNS	Simple Notification Service	A managed, push-based publisher-subscriber (pub/sub) service used for event fan-out distribution.
DLQ	Dead-Letter Queue	A target queue used to capture messages that fail processing repeatedly, keeping the main queue unblocked.
ESM	Event Source Mapping	An AWS-managed poller that queries resources (like SQS) and invokes a Lambda function with batch records.
Visibility Timeout	Visibility Timeout	The duration of time a message is hidden from other consumers while one consumer is processing it.
Redrive Policy	Redrive Policy	The configuration settings mapping a source queue to a DLQ, specifying the maxReceiveCount retry threshold.

Acronym / Term	Full Name	Definition / Cloud Role
Step Functions	AWS Step Functions	A visual orchestration service used to coordinate multi-step workflows using State Machines.
EventBridge	Amazon EventBridge	A serverless event bus that routes events using rules, or coordinates scheduled triggers.

Appendix B: Week 4 Troubleshooting & Debugging

Error Message / Issue	Likely Cause	Step-by-Step Resolution
AccessDenied when Lambda writes to SQS	The execution IAM role attached to the producer Lambda lacks permissions to perform sqs:SendMessage.	Verify your IAM policy resource bindings in main.tf. Ensure the role contains a policy permitting SendMessage on the target SQS queue ARN.
SQS messages stuck "In Flight"	The Visibility Timeout on the queue is shorter than the Lambda function's timeout, causing double-processing.	Edit the SQS queue resource settings in main.tf. Set the visibility_timeout_seconds to at least 6 times the timeout of your consumer Lambda function.
Messages instantly moving to DLQ	The Redrive Policy maxReceiveCount is set to 1, and the consumer function is erroring on	1. Check your consumer logs in CloudWatch to fix the code crash. 2. Increase the maxReceiveCount in your redrive policy (e.g., to 3 or 5) to allow retry

Error Message / Issue	Likely Cause	Step-by-Step Resolution
	execution.	attempts.
SNS Email notifications not received	1. The subscription was not confirmed. 2. The email was marked as spam.	1. Open the inbox of the email used for subscription. 2. Search for a mail with sender AWS Notifications and click the Confirm Subscription link.
401 Unauthorized on API Gateway	The API Gateway Cognito authorizer is expecting an ID token, but an Access token (or a malformed token) was passed.	Ensure that you pass the base64-encoded Cognito ID token (not the Access token) inside the Authorization header prefix.